

Partie 2 : Manuel d' Ingénierie de l' Animation Haut de Gamme

Ce document est le guide de référence pour l'implémentation technique du stack d'animation. Il définit les standards de code, les architectures de projet et les méthodes d'intégration pour transformer des concepts créatifs en réalités numériques fluides et performantes.

1. GSAP (GREENSOCK) : LE CŒUR DU MOUVEMENT

GSAP n'est pas une simple bibliothèque, c'est un moteur de rendu temporel. Pour un site haut de gamme, l'utilisation doit être structurée autour des **Timelines**.

1.1. Architecture des Timelines

L'équipe doit bannir les animations isolées au profit de séquences orchestrées.

- **Master Timelines** : Créer une timeline parente pour chaque section. Cela permet de contrôler l'entrée, le maintien et la sortie des éléments de manière synchrone.
- **Labels Temporels** : Utiliser `tl.addLabel('nom', time)` pour créer des points d'ancrage. Cela facilite la synchronisation d'événements complexes (ex: déclencher un son exactement quand un texte finit de s'afficher).
- **Contrôle du Flux** : Utiliser les méthodes `play()`, `pause()`, `reverse()` et `seek()` pour lier l'animation à des événements utilisateurs (clics, changements d'onglets).

1.2. ScrollTrigger : L'Interactivité au Défilement

Le scroll ne doit pas seulement déplacer la page, il doit piloter l'animation.

- **Scrubbing** : Lier la progression de l'animation à la position du scroll (`scrub: true`). Plus l'utilisateur scrolle vite, plus l'animation est rapide.
- **Pinning** : Fixer un élément à l'écran (`pin: true`) pendant que son contenu interne s'anime. Indispensable pour les démonstrations de produits par étapes.
- **Anticipation (Start/End)** : Définir des points de déclenchement précis (ex: `start: "top 80%"`). L'animation doit commencer avant que l'élément ne soit au centre de l'écran pour éviter les apparitions brusques.

1.3. Plugins Spécialisés

- **SplitText** : Pour animer chaque lettre ou mot individuellement sans casser l'accessibilité (SEO).
 - **Flip Plugin** : Pour des transitions d'état fluides entre deux mises en page radicalement différentes (ex: une vignette qui devient une bannière plein écran).
-

2. INTÉGRATION 3D : THREE.JS ET SPLINE

Le haut de gamme exige de la profondeur. La 3D doit être interactive et légère.

2.1. Workflow avec Spline

Spline est l'outil de choix pour les objets 3D interactifs rapides.

- **State Machines** : Définir les états (survol, clic, repos) directement dans Spline. L'équipe technique doit ensuite piloter ces états via l'API JS de Spline.
- **Variables de Contrôle** : Lier la position de la souris aux coordonnées X/Y d'un objet 3D pour créer un effet de regard (LookAt).
- **Optimisation Export** : Utiliser le format `Spline Runtime` pour conserver une interactivité totale avec un poids de fichier minimal.

2.2. Three.js pour le Sur-Mesure

Pour les expériences immersives totales, Three.js est requis.

- **Gestion de la Scène** : Utilisation de `PerspectiveCamera` pour un réalisme accru.
 - **Matériaux Avancés** : Utiliser `MeshPhysicalMaterial` pour simuler des textures réelles (verre, métal, plastique) avec des reflets dynamiques.
 - **Lumières Dynamiques** : Implémenter des `PointLight` ou `SpotLight` qui suivent le curseur de l'utilisateur pour créer une interaction lumineuse directe.
-

3. RIVE : L'ANIMATION VECTORIELLE INTELLIGENTE

Rive remplace avantageusement Lottie pour les animations complexes et interactives.

3.1. Les State Machines de Rive

- **Inputs Booléens et Numériques** : L'animation change selon des variables envoyées par le code (ex: `is_loading = true` déclenche une transition vers l'état de chargement).
- **Transitions Smooth** : Rive gère l'interpolation entre les états, garantissant qu'une animation ne "saute" jamais, même si l'utilisateur change d'avis en plein milieu.

3.2. Performance Vectorielle

- **Poids Plume** : Les fichiers Rive sont souvent 10x plus légers que les GIFs ou les vidéos.
 - **Indépendance de la Résolution** : Pas de pixellisation, même sur les écrans 8K ou lors d'un zoom extrême.
-

4. FLUIDITÉ ET DÉFILEMENT (SMOOTH SCROLL)

Le "feeling" du site dépend de la qualité du défilement.

4.1. Lenis Scroll

Lenis est actuellement le standard pour le défilement fluide.

- **Inertie Personnalisée** : Régler la friction pour que le scroll s'arrête de manière soyeuse, pas brusque.
 - **Synchronisation GSAP** : Lenis doit être couplé à `ScrollTrigger.refresh()` pour que les positions des animations soient recalculées en temps réel.
 - **Support Mobile** : S'assurer que le "smooth scroll" ne dégrade pas l'expérience tactile native.
-

5. MÉTHODOLOGIE DE DÉVELOPPEMENT CRÉATIF

5.1. La Règle du "60 FPS or Nothing"

Toute animation qui fait chuter le framerate doit être optimisée ou retirée.

- **Layers de Rendu** : Utiliser `transform: translateZ(0)` pour forcer la création d'une couche GPU.
- **Debouncing** : Ne jamais lier des calculs lourds directement à l'événement `scroll` ou `mousemove`. Utiliser `requestAnimationFrame`.

5.2. Accessibilité (A11y)

- **Prefers-Reduced-Motion** : Le code doit détecter cette préférence système et désactiver automatiquement les mouvements non essentiels.
- **Fallback Statique** : Si le WebGL échoue à se charger, une image de remplacement haute définition doit être affichée immédiatement.

5.3. Debugging d'Animation

- **GSDevTools** : Utiliser l'interface de contrôle de GSAP pendant le développement pour tester les animations au ralenti et trouver les imperfections de timing.
- **Stats.js** : Afficher un compteur de FPS en temps réel lors des phases de test sur mobile.

RÉFÉRENCES D'IMPLÉMENTATION

- **GSAP Learning Center** : <https://greensock.com/learning/>
- **Three.js Journey** : <https://threejs-journey.com/>
- **Rive Documentation** : <https://help.rive.app/>
- **Lenis GitHub** : <https://github.com/studio-freight/lenis>

Ce document doit être scrupuleusement suivi pour garantir l'intégrité technique de la vision créative.

Partie 2 (V2) : Ingénierie de l' Animation de Prestige - Détails Avancés

Ce document approfondit les concepts de la Partie 2 pour les développeurs seniors. Il traite de la gestion de la mémoire, de l'interopérabilité des systèmes et des standards de rendu physique.

1. ARCHITECTURE GSAP AVANCÉE

1.1. GSAP Context & Cleanup (Frameworks React/Vue)

Dans les applications modernes, la gestion de la mémoire est critique.

- **gsap.context()** : Utilisation systématique de `gsap.context()` pour regrouper toutes les animations d'un composant. Cela permet un nettoyage (revert) automatique lors du démontage du composant, évitant les fuites de mémoire et les animations fantômes.
- **MatchMedia** : Utiliser `gsap.matchMedia()` pour définir des timelines différentes selon la taille de l'écran (Desktop vs Mobile) dans un seul bloc de code propre.

1.2. Custom Easing & Physics

- **CustomEase** : Créer des courbes de Bézier propriétaires pour la marque. Ne pas se contenter de `power2.out`, mais définir une signature de mouvement unique.
 - **GSAP InertiaPlugin** : Pour les interactions de glisser-déposer ou les sliders, utiliser ce plugin pour simuler une friction réelle basée sur la vitesse du lancer de l'utilisateur.
-

2. RENDU 3D DE HAUTE PRÉCISION (THREE.JS)

2.1. PBR (Physically Based Rendering)

Pour que la 3D paraisse "luxe", elle doit interagir avec la lumière comme dans la réalité.

- **Environment Maps (HDR)** : Utiliser des fichiers HDR pour l'éclairage global. Cela permet d'obtenir des reflets réalistes sur les surfaces métalliques ou en verre.
- **Roughness & Metalness** : Paramétrage précis des textures pour que la lumière "accroche" les bords des objets de manière organique.
- **Anisotropie** : Pour les textures comme le métal brossé, activer l'anisotropie pour un rendu ultra-détaillé sous tous les angles.

2.2. Raycasting et Interactivité DOM-3D

- **Raycaster** : Implémenter un système de détection de collision entre la souris et les objets 3D. Un clic sur une zone spécifique de l'objet 3D doit pouvoir déclencher une action dans le DOM (ex: ouvrir une fiche produit HTML).

- **HTML Overlay (CSS3DRenderer)** : Intégrer des éléments HTML réels (texte, boutons) directement dans la scène 3D pour qu'ils suivent les mouvements de caméra tout en restant sélectionnables et indexables par Google.
-

3. OPTIMISATION DU PIPELINE DE RENDU

3.1. Draw Call Reduction

Chaque objet envoyé au GPU coûte de la performance.

- **InstancedMesh** : Si vous avez 1000 particules ou objets identiques, utilisez `InstancedMesh` pour les dessiner en un seul appel (Draw Call).
- **Texture Merging** : Combiner plusieurs petites textures en un seul "Atlas" pour minimiser les changements d'état du GPU.

3.2. Web Workers pour la Physique

Si le site inclut des simulations physiques (gravité, collisions complexes) :

- **Offloading** : Déplacer les calculs de physique (bibliothèques comme `Ammo.js` ou `Cannon-es`) dans un Web Worker. Cela garantit que l'interface reste réactive à 60 FPS même pendant des calculs lourds.
-

4. STANDARDS DE QUALITÉ VISUELLE

4.1. Anti-Aliasing (Anticrénelage)

- **SMAA vs MSAA** : Utiliser le `SMAAPass` en post-processing pour un lissage des bords parfait sans l'impact de performance du MSAA natif sur les écrans haute résolution.
- **Pixel Ratio** : Limiter le `pixelRatio` à 2 pour les écrans Retina afin d'éviter de surcharger le GPU inutilement sur les écrans 4K/5K.

4.2. Color Management

- **sRGB Encoding** : S'assurer que toutes les textures et le rendu final utilisent l'encodage sRGB pour des couleurs fidèles et vibrantes, évitant l'aspect "lavé" des rendus WebGL par défaut.
-

RÉFÉRENCES SENIOR

- **GSAP Architecture Patterns** : <https://greensock.com/react/>
- **Three.js Performance Tips** : <https://discoverthreejs.com/tips-and-tricks/>
- **WebGL Best Practices** : https://www.khronos.org/webgl/wiki/WebGL_Best_Practices

Ce document complète la première version pour offrir une vision technique sans faille.